SimpleImputer:
"A preprocessing technique used to replace missing values in a dataset using strategies such as mean, median, most frequent value, or a constant value."

Pipeline:
"A Pipeline is a sequence of data preprocessing and machine learning steps executed automatically in a fixed order, making code cleaner, reusable, and less error-prone."

```
num_pipeline = Pipeline([
    ('impute', SimpleImputer()),
    ('scale', StandardScaler())
])
```

means:

```
Numerical Data
      ↓
Fill Missing Values
      ↓
Scale Features
      ↓
Ready for Machine Learning Model
```

A Pipeline can contain:

- 1 preprocessing step
- 2 preprocessing steps
- Many preprocessing steps
- A preprocessing step + a machine learning model

Example 1: Two Steps

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

pipeline = Pipeline([
    ('imputer', SimpleImputer()),
    ('scaler', StandardScaler())
])
```

Flow:

```
Data
 ↓
Imputer
 ↓
Scaler
 ↓
Output
```

Example 2: Three Steps

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, PolynomialFeatures

pipeline = Pipeline([
    ('imputer', SimpleImputer()),
    ('scaler', StandardScaler()),
    ('poly', PolynomialFeatures(degree=2))
])
```

Flow:

```
Data
 ↓
Imputer
 ↓
Scaler
 ↓
Polynomial Features
 ↓
Output
```

Example 3: Preprocessing + Model

Most common usage.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

pipeline = Pipeline([
    ('imputer', SimpleImputer()),
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

Flow:

```
Data
 ↓
Imputer
 ↓
Scaler
 ↓
Linear Regression
 ↓
Prediction
```

Can We Add Multiple Models in One Pipeline?

Directly NO.

This is invalid:

```
Pipeline([
    ('lr', LinearRegression()),
    ('rf', RandomForestRegressor())
])
```

Reason:

- LinearRegression gives predictions.
- RandomForest expects training features, not predictions.

Then How Can Multiple Models Be Used?

There are special techniques.

1. Voting

```
from sklearn.ensemble import VotingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

model = VotingRegressor([
    ('lr', LinearRegression()),
    ('rf', RandomForestRegressor())
])
```

Both models predict.

Final answer = average prediction.

---

2. Stacking

```
from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

model = StackingRegressor(
    estimators=[
        ('lr', LinearRegression()),
        ('rf', RandomForestRegressor())
```

```
    ],
    final_estimator=LinearRegression()
)
```

Flow:

```
Data
 ↓
Linear Regression
 ↓
Random Forest
 ↓
Meta Model
 ↓
Final Prediction
```

Complete Pipeline Example

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor

pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('model', RandomForestRegressor())
])

pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)
```

Flow:

```
Raw Data
   ↓
Fill Missing Values
   ↓
Scale Features
   ↓
Random Forest Model
   ↓
Predictions
```

Can a Pipeline contain multiple models?

Answer: A standard Scikit-Learn Pipeline can have many preprocessing steps but only one final machine learning model. To combine multiple models, techniques like Voting, Bagging, Boosting, or Stacking are used.

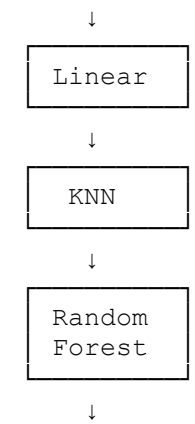
multiple models connected together, you usually use **Stacking**, **Voting**, or **Bagging** rather than a normal Pipeline.

1. Stacking (Models Connected Sequentially)

multiple models make predictions, and another model learns from those predictions.

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import StackingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import r2_score

# Load data
data = fetch_california_housing()
X = data.data
y = data.target

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Base models
base_models = [
    ('lr', LinearRegression()),
    ('knn', KNeighborsRegressor()),
    ('rf', RandomForestRegressor())
]

# Final model
stack_model = StackingRegressor(
    estimators=base_models,
    final_estimator=LinearRegression()
)

# Train
stack_model.fit(X_train, y_train)

# Predict
y_pred = stack_model.predict(X_test)

print("R2 Score:", r2_score(y_test, y_pred))
```

Flow:



```
 Meta Model
     ↓
 Final Output
```

---

2. Voting Regressor (Models Work Together)

All models predict independently.

Final prediction = Average of predictions.

```
from sklearn.ensemble import VotingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor

model = VotingRegressor([
    ('lr', LinearRegression()),
    ('knn', KNeighborsRegressor()),
    ('rf', RandomForestRegressor())
])

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("R2 Score:", r2_score(y_test, y_pred))
```

Example:

```
Linear Regression → 200000
KNN               → 210000
Random Forest     → 220000

Final Prediction
= (200000+210000+220000)/3
= 210000
```

---

3. Pipeline + Multiple Models Comparison

This is very common in industry.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor

models = {
    "Linear": LinearRegression(),
    "RandomForest": RandomForestRegressor(),
    "KNN": KNeighborsRegressor()
}

for name, model in models.items():
```

```
    pipe = Pipeline([
        ('imputer', SimpleImputer()),
        ('scaler', StandardScaler()),
        ('model', model)
    ])

    pipe.fit(X_train, y_train)

    score = pipe.score(X_test, y_test)

    print(name, ":", score)
```

This creates **3 separate pipelines**, each ending with a different model.

Why create a new Pipeline in every loop?

Because each model needs the **same preprocessing**:

```
Raw Data
   ↓
SimpleImputer
   ↓
StandardScaler
   ↓
Different Model
```

For example:

```
Pipeline 1
Imputer → Scaler → Linear Regression

Pipeline 2
Imputer → Scaler → Random Forest

Pipeline 3
Imputer → Scaler → KNN
```

To see predictions too

```
for name, model in models.items():

    pipe = Pipeline([
        ('imputer', SimpleImputer()),
        ('scaler', StandardScaler()),
        ('model', model)
    ])

    pipe.fit(X_train, y_train)

    y_pred = pipe.predict(X_test)

    print("\n", name)
    print("R2 Score:", pipe.score(X_test, y_test))
    print("First 5 Predictions:", y_pred[:5])
```

Example output:

```
Linear
R2 Score: 0.65
First 5 Predictions:
[210000 185000 320000 150000 275000]

RandomForest
R2 Score: 0.82
First 5 Predictions:
[220000 190000 330000 145000 280000]

KNN
R2 Score: 0.74
First 5 Predictions:
[205000 180000 310000 155000 270000]
```

---

4. Pipeline + Stacking (Advanced)

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import StackingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor

stack_model = StackingRegressor(
    estimators=[
        ('rf', RandomForestRegressor()),
        ('knn', KNeighborsRegressor())
    ],
    final_estimator=LinearRegression()
)

pipeline = Pipeline([
    ('imputer', SimpleImputer()),
    ('scaler', StandardScaler()),
    ('stacking', stack_model)
])

pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)
```

Flow:

```
Raw Data
   ↓
Imputer
   ↓
Scaler
   ↓
Random Forest
   ↓
KNN
   ↓
```

```
Linear Regression
   ↓
Prediction
```

- **Pipeline** = sequence of preprocessing steps + usually one final model.
- **Voting** = many models vote together.
- **Stacking** = many models feed into another model.
- **Bagging** = same model trained many times on different samples.
- **Boosting** = models are trained one after another, correcting previous errors.